

Asymptotic Notation

Algorithms and Complexity - Computer Science

1. Why complexity matters

Measuring an algorithm by wall-clock time describes the machine as much as the method. Counting how the number of elementary operations grows with the input size is a property of the algorithm itself, and survives a change of hardware, language or compiler.

2. The three bounds

Big-O states an upper bound: $f(n) = O(g(n))$ when $f(n) \leq c \cdot g(n)$ beyond some n_0 . Big-Omega states the matching lower bound, and Big-Theta applies when the two agree, pinning the growth rate exactly. Quote the tightest bound you can prove - binary search is $O(n)$, but saying so is useless when it is $\Theta(\log n)$.

3. Reading the common rates

Constant $O(1)$ covers array indexing. Logarithmic $O(\log n)$ arises when the input halves each step. Linear $O(n)$ examines each element once, linearithmic $O(n \log n)$ is the comparison-sorting bound, quadratic $O(n^2)$ comes from nested loops over one collection, and exponential $O(2^n)$ from exhaustive search over subsets.

4. Worst, average and amortised

Worst case bounds every input and is the usual quotation. Average case assumes a distribution, which is why quicksort is called $O(n \log n)$ despite a quadratic worst case. Amortised cost averages over a sequence: a doubling dynamic array copies occasionally, yet costs constant time per insertion overall.